

PythonPowerFlow: OPF Comparison Framework

Short Technical Report

David Jose Florez Rodriguez

1 Introduction

PythonPowerFlow is a framework for comparing convex relaxations of AC Optimal Power Flow (OPF) against local and heuristic solvers on standard test networks. The AC OPF minimises generation cost subject to the nonlinear AC power flow equations and operating limits:

$$\begin{aligned} \min \quad & \sum_g c_g P_g \\ \text{s.t.} \quad & P_i = \sum_{g \in i} P_g - P_{\text{load},i}, \quad Q_i = \sum_{g \in i} Q_g - Q_{\text{load},i} \\ & V_{\min} \leq |V_i| \leq V_{\max}, \quad P_g^{\min} \leq P_g \leq P_g^{\max} \end{aligned}$$

The power balance is nonlinear ($|V_i||V_k| \cos \theta_{ik}$, $|V_i||V_k| \sin \theta_{ik}$), making AC OPF non-convex and NP-hard. The **project SOCP** relaxation replaces the rank-1 voltage constraint with a convex second-order cone, providing a tractable global lower bound. The framework runs four solvers, records nine comparison metrics, and reports exactness diagnostics (τ , δ) per test case.

2 Solvers

2.1 Circuits NR

Full Newton–Raphson AC power flow. Solves the nonlinear power-balance equations from a given starting point; finds one locally valid AC solution. Highly sensitive to initialisation on meshed networks. Returns no objective value (power-flow only, no dispatch optimisation).

2.2 project DC OPF

Linear DC OPF formulated as an LP. Approximates $|V_i| \approx 1$ pu and neglects reactive power and losses. Susceptance: $b_{ij} = x/(r^2 + x^2)$. Returns zero reactive flows. Provides a fast, loose lower bound on the AC OPF cost.

2.3 project SOCP

SOCP relaxation of AC OPF (Jabr 2006/2007). Introduces *lifted variables*:

$$W_{ii} = |V_i|^2, \quad W_{ij}^R = \text{Re}(V_i V_j^*), \quad W_{ij}^I = \text{Im}(V_i V_j^*)$$

Power balance becomes *linear* in these variables:

$$P_i = G_{ii} W_{ii} + \sum_{k \neq i} (G_{ik} W_{ik}^R + B_{ik} W_{ik}^I)$$

Physical voltages require $\mathbf{W} = \mathbf{V}\mathbf{V}^H$ (rank-1). The SOCP relaxes each 2×2 principal submatrix to:

$$\|(2W_{ij}^R, 2W_{ij}^I, W_{ii} - W_{jj})\|_2 \leq W_{ii} + W_{jj}$$

equivalent to $|W_{ij}|^2 \leq W_{ii} W_{jj}$ (Cauchy–Schwarz relaxation of the physical equality). Since the SOCP feasible

set *contains* the AC OPF feasible set:

$$\text{project SOCP objective} \leq \text{AC OPF objective}$$

Solved via a CLARABEL→GUROBI→SCS cascade. Runs in *PF mode* (fix dispatch, compare voltages) or *OPF mode* (minimise cost); PV bus voltages are free in OPF mode.

2.4 pandapower AC OPF

Interior-point NLP (`pp.runopf`). Finds a locally optimal AC solution; used as a ground-truth reference on WB5 and Case22loop variants.

Solver summary:

Solver	Type	Bound
Circuits NR	NR AC PF	None (local)
project DC OPF	LP	Lower bound
project SOCP	SOCP (convex)	Lower bound
pandapower AC OPF	NLP (local)	None (local)

3 Metrics

The interactive CLI (`my_opfs.py`) exposes nine metrics:

1. **Bus voltage magnitudes** $|V_i|$ (pu)
2. **Voltage angles** θ_i (deg)
3. **Branch active flows** P_{ij} (pu)
4. **Branch reactive flows** Q_{ij} (pu; zero for DC)
5. **Objective value** $\sum_g c_g P_g$ (\$/h)
6. **Convergence status**
7. **Solve time** (s)
8. **SOCP branch tightness** τ_k
9. **Loop residuals** δ_k (deg)

3.1 Branch tightness τ_k

$$\tau_k = \frac{(W_{ij}^R)^2 + (W_{ij}^I)^2}{W_{ii} \cdot W_{jj}} \in [0, 1]$$

$\tau_k = 1$: the SOC constraint is active; the submatrix is locally rank-1. $\tau_k < 1$: the relaxation is slack — a *certificate of inexactness*.

3.2 Loop residuals δ_k

A BFS spanning tree assigns angles θ_i from the slack bus. For each non-tree branch $k = (i, j)$, let $\phi_{ij}^{\text{dir}} = \arctan(W_{ij}^I/W_{ij}^R)$ be the phase implied by the SOCP solution, and $\phi_{ij}^{\text{tree}} = \theta_i - \theta_j$ the phase predicted by the tree path. Then:

$$\delta_k = \phi_{ij}^{\text{dir}} - \phi_{ij}^{\text{tree}} \quad (\text{wrapped to } (-\pi, \pi])$$

$\delta_k \neq 0$ signals that phase assignments around that fundamental cycle are *inconsistent*: no global phasor set can produce this W matrix, even when all $\tau_k = 1$.

Interpretation:

$\tau_k=1$ all k	$\delta_k=0$ all loops	Conclusion
No	—	Provably inexact
Yes	Yes	SOC exact; AC-feasible
Yes	No	Inexact (phase inconsistency)

4 Test Cases

Case	Buses	Lines	Key property
IEEE 14-bus	14	20	Meshed; $\tau \approx 1$, $ \delta \leq 4.3^\circ$
WB5	5	6	41% SOCP gap
C22 (uniform)	22	22	Ring; SOCP exact
C22 (asymm.)	22	22	Two AC optima; SOCP exact

4.1 IEEE 14-bus

Standard MATPOWER case; 5 generators, 3 transformers. project SOCP runs in *PF mode* (compared to Circuits NR as ground truth). Voltage bounds $[0.5, 1.5]$ pu (Q-limits widened: the model omits the Bus 9 shunt capacitor). SOCP is tight ($\tau \approx 1$) yet loop residuals reach 4.3° — illustrating that $\tau = 1$ everywhere does *not* certify AC-feasibility on a meshed network.

4.2 WB5

5-bus meshed network (Bukhsh et al. 2013). Bus 1: \$4/MWh; Bus 5: \$1/MWh. The cheap generator is reachable only via two high-impedance lines ($r = 0.55$, $x = 0.90$ pu), creating a non-convex bottleneck. Voltage bounds $[0.5, 1.5]$ pu. The canonical example of a non-zero SOCP duality gap.

4.3 Case22loop — uniform costs

22-bus ring; all 11 generators at \$2/MWh. One branch beyond the spanning tree creates one loop degree of freedom. Voltage bounds $[0.95, 1.05]$ pu. SOCP is exact: $\tau_k \approx 1$, $\delta = 0^\circ$.

4.4 Case22loop — asymmetric costs

Same ring; buses 1, 3, ..., 11 (cheap arc) at \$1/MWh; buses 13, ..., 21 (expensive arc) at \$4/MWh. The global optimum loads only the cheap arc; flat-start Circuits NR converges to a balanced but expensive solution. SOCP remains exact ($\delta = 0.74^\circ \approx 0$).

5 Results

5.1 WB5 duality gap

Solver	Obj. (\$/h)	Note
project DC OPF	325	Linear LB; no reactive/losses
project SOCP	740	Convex LB; Bus 5 at $V_{\max} = 1.5$ pu
pandapower AC OPF	1,256	True local AC optimum
Duality gap	515 (41%)	$\delta_{L04-05} = 1.58^\circ$

All SOC constraints are tight ($\max(1-\tau_k) \approx 3 \times 10^{-9}$) yet loop residuals are nonzero. The SOCP exploits freedom to assign inconsistent phases around the two fundamental cycles, reaching a cost that no physical voltage assignment can realise.

5.2 Case22loop: exactness and multiple optima

Solver	Uniform (\$/h)	Asymm. (\$/h)
project DC OPF	4,494	2,244
project SOCP	4,541	2,464
pandapower AC OPF	4,541	2,464
Circuits NR (flat)	4,544	5,336

On the asymmetric ring, Circuits NR (flat start) finds the balanced operating point at 5,336 \$/h; project SOCP and pandapower AC OPF find the globally optimal dispatch at 2,464 \$/h — a **116% cost difference** from routing power exclusively through the cheaper arc.

5.3 SOCP diagnostics summary

Case	$\max(1 - \tau_k)$	$\max \delta_k $	Gap
IEEE 14-bus	5×10^{-6}	4.3°	≈ 0
WB5	3×10^{-9}	1.58°	41%
C22 uniform	~ 0	0.0°	≈ 0
C22 asymm.	3×10^{-12}	0.74°	≈ 0

Key takeaway. $\tau_k \approx 1$ alone is insufficient to certify exactness. WB5 has tight SOC constraints everywhere yet a 41% duality gap — revealed only by its nonzero loop residuals. The ring cases satisfy both $\tau \approx 1$ *and* $\delta \approx 0^\circ$, confirming their SOCP solutions are globally AC-feasible and optimal.

On *radial* networks, the SOC relaxation is provably exact (Jabr 2006, Low 2014): no fundamental cycles exist, so loop residuals vanish, and $\tau_k = 1$ everywhere guarantees $\mathbf{W} = \mathbf{V}\mathbf{V}^H$ (rank-1). The SOCP yields the global AC OPF optimum at the cost of a single convex solve.

Solvers: project SOCP via CLARABEL/GUROBI/SCS (cvxpy); project DC OPF via cvxpy LP; Circuits NR and pandapower AC OPF via pandapower/PYPOWER. All powers in per-unit on a 100 MVA base.

Using my_opfs.py

Interactive OPF Solver Comparison CLI

<https://github.com/DavidLikesLearning/PythonPowerFlow/tree/DavidProject>

Quick Start

```
python3 my_opfs.py
```

Launches a five-step interactive session. All heavy solver imports happen after the prompts, so the menu appears instantly. Press Ctrl-C at any prompt to exit.

Step 1 — Choose a Grid

- 1. IEEE 14-bus** — 14-bus meshed network. SOCP runs in PF mode (compare to NR). $\tau \sim 1$ everywhere but loop residuals up to 4.3° — shows $\tau = 1$ does not imply exactness.
- 2. WB5** — 5-bus Bukhsh 2013 network. SOCP runs in OPF mode. 41% duality gap — canonical non-exact SOCP example.
- 3. Case22loop (uniform)** — 22-bus ring, all generators at \$2/MWh. SOCP exact (zero duality gap, loop residual = 0°).
- 4. Case22loop (asymmetric)** — 22-bus ring, cheap arc \$1/MWh vs expensive arc \$4/MWh. Two distinct local AC optima; SOCP still exact.

Step 2 — Choose Solvers

Enter comma-separated numbers (e.g. 1, 3, 5) or press Enter to run all five:

- 1. Circuits NR — Newton-Raphson AC power flow (feasibility, no objective)
- 2. Project DC OPF — cvxpy LP, lossless DC approximation
- 3. Project SOCP — Jabr SOCP relaxation (PF or OPF mode, fixed per grid)
- 4. Panda DC OPF — pandapower DC OPF (uses $1/x$ susceptance)
- 5. Panda AC OPF — PYPOWER interior-point AC OPF (slowest, most accurate)

Step 3 — Generator Costs

Default costs are shown for the chosen grid. Enter **Y** to accept them, or **N** to supply custom values in one of two ways:

Comma-separated list — one number per generator in the displayed order, e.g. 4, -1 for WB5 (GenSlack, GenPV).
CSV file — either a single-column file (costs in order) or a two-column `gen_name, cost` file with an optional header row.

Step 4 — Metrics to Display

Enter comma-separated numbers or press Enter for all nine:

- 1. Voltage magnitudes (pu)
- 2. Voltage angles ($^\circ$)
- 3. Branch P flows — from-end active power (pu)
- 4. Branch Q flows — from-end reactive power (pu, zero for DC solvers)
- 5. Objective value (\$/h) — N/A for NR
- 6. Convergence status
- 7. Solve time (s)
- 8. SOCP tightness τ per branch — always from the SOCP result

- 9. Loop residuals (°) — phase inconsistency around fundamental cycles

Step 5 — Save Outputs

CSV — four-section comparison file: (1) solver summary, (2) bus voltages, (3) branch flows, (4) SOCP tightness and loop residuals. Default filename is `<grid>_comparison.csv`.

Plots — PNG figures saved to a chosen directory: `fig_voltages.png`, `fig_angles.png`, `fig_branch_p.png`, `fig_branch_q.png`, `fig_objectives.png`, `fig_tightness.png`.

Example Sessions

Check the WB5 duality gap

Step 1: 2 · Step 2: 3,5 · Step 3: Y · Step 4: 5,8,9 · Step 5: Y / N

Verify SOCP vs NR on IEEE 14-bus

Step 1: 1 · Step 2: 1,3 · Step 3: Y · Step 4: 1,6,7 · Step 5: N / N

Run asymmetric ring with all solvers

Step 1: 4 · Step 2: [Enter] · Step 3: Y · Step 4: [Enter] · Step 5: Y / Y

Voltage bounds are set automatically per grid: WB5 uses [0.5, 1.5] pu (natural operating point is below 0.95 pu); all other grids use [0.95, 1.05] pu.

PythonPowerFlow — Interactive OPF Demo

WB5 Five-Bus Case · All Solvers · All Metrics

<https://github.com/DavidLikesLearning/PythonPowerFlow/tree/DavidProject>

PythonPowerFlow compares five OPF solvers on standard test networks using an interactive CLI (**my_opfs.py**). The WB5 network (Bukhsh et al. 2013) is the canonical example of a non-exact SOCP relaxation: the cheap generator at Bus05 (\$1/MWh) is only reachable via two high-impedance lines, creating a 41% duality gap between the SOCP lower bound (\$740/h) and the true AC optimum (\$1,256/h).

Solvers run: Project SOCP (OPF) · Project DC OPF · Circuits NR · Panda DC OPF · Panda AC OPF

Bus Voltage Magnitudes

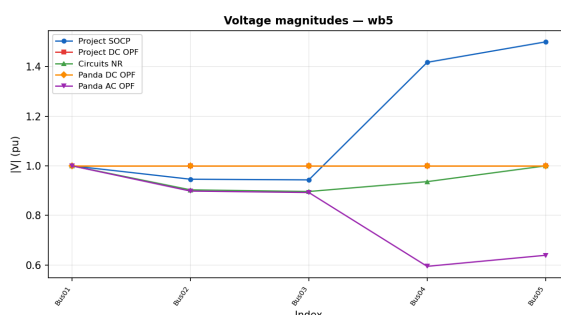


Figure 1. Bus voltage magnitudes (pu). SOCP pushes Bus04–05 to the 1.5 pu upper bound; Panda AC OPF finds the physical solution (Bus04 = 0.59 pu, Bus05 = 0.64 pu); DC solvers fix all voltages at 1.0 pu.

Bus Voltage Angles

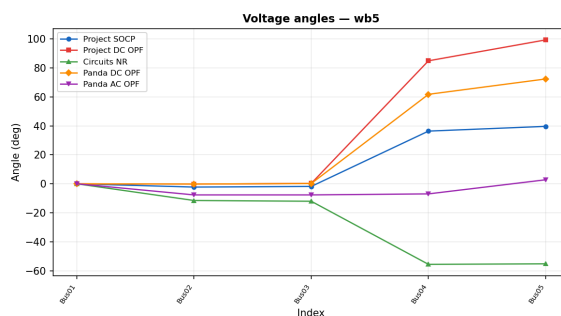


Figure 2. Voltage angles (°). SOCP and DC OPF show unrealistically large spreads (Bus05 up to +99°) reflecting relaxed voltages. NR and Panda AC OPF produce physically consistent negative angles.

Why do Project DC OPF and Panda DC OPF give different angles? Both use the same lossless DC approximation ($V = 1$ pu, linearised power balance) but differ in how they compute the line susceptance b_{ij} . Project DC OPF uses $b_{ij} = x / (r^2 + x^2)$ — the imaginary part of the series admittance $y = 1 / (r + jx)$, which is exact for the DC model. Panda DC OPF uses $b_{ij} = 1 / x$ — ignoring resistance entirely. On WB5's high-impedance lines ($r = 0.55$, $x = 0.90$ pu) the ratio $r/x = 0.61$, so the two susceptance values differ significantly: $b = 0.90 / (0.55^2 + 0.90^2) = 0.865$ pu vs. $1/0.90 = 1.111$ pu. Pandapower's larger susceptances make the network appear stiffer, which compresses the angle spread (Bus05: +72° vs. +99° for Project DC OPF). Both are valid DC approximations; the discrepancy vanishes on low- r/x lines.

Duality Gap & SOCP Tightness

Branch tightness $\tau = |W_{ij}|^2 / (W_{ii} \cdot W_{jj})$ measures SOC slack per branch. $\tau = 1$ everywhere is necessary but *not sufficient* for exactness on meshed networks — loop residuals must also be zero.

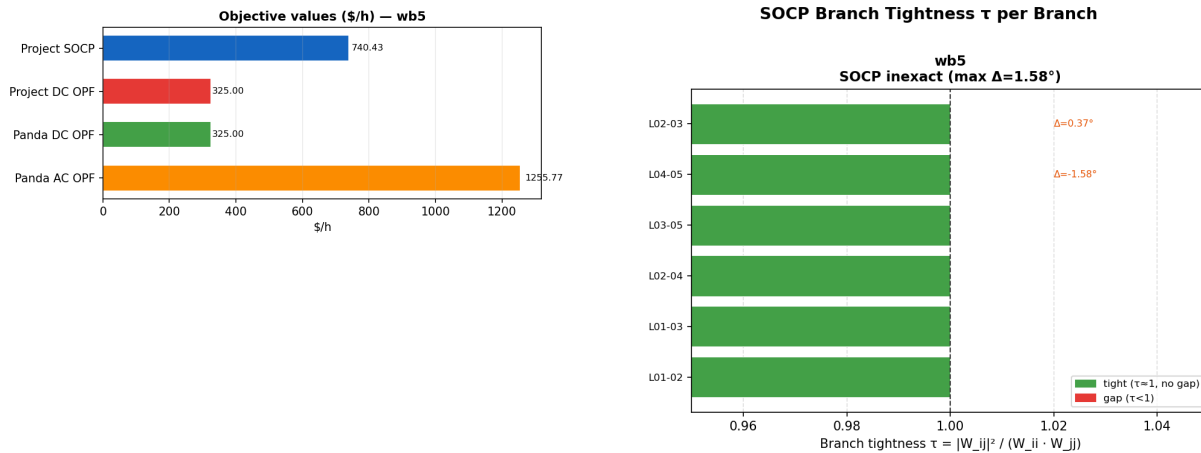


Figure 3 (left). Objective values (\$/h): SOCP (\$740) and DC OPF (\$325) are lower bounds; Panda AC OPF (\$1,256) is the true AC optimum — 41% duality gap. Figure 4 (right). Per-branch SOC tightness τ : all 6 branches reach $\tau = 1$ (gap $< 3e-9$) — the relaxation is tight everywhere.

WB5 key result — $\tau = 1$ everywhere, yet 41% duality gap:

All 6 SOC constraints are active, but loop residuals (L04-05: -1.58° , L02-03: $+0.37^\circ$) reveal phase inconsistency around the two fundamental cycles. The W matrix is not globally rank-1 — the SOCP optimum is AC-infeasible. This is the canonical illustration that $\tau = 1$ is **necessary but not sufficient** for SOCP exactness on meshed networks.

Branch Active Power Flows

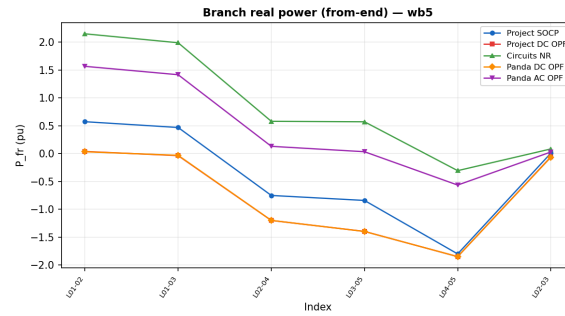


Figure 5. Active power P_{from} (pu). SOCP and DC OPF show negative flows on L02-04 and L03-05 (cheap generator pulling power backwards through high-impedance lines). NR and AC OPF show positive flows at the physical operating point.

Branch Reactive Power Flows

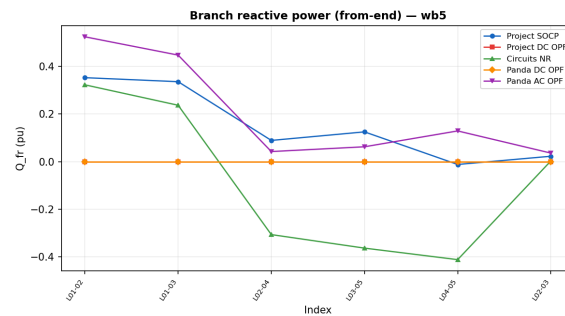


Figure 6. Reactive power Q_{from} (pu). DC solvers return $Q = 0$ by definition. SOCP and AC solvers show substantial reactive flows on L01-02 and L01-03 (slack bus supplies reactive demand). Circuits NR shows negative Q on L02-04 and L03-05 at the physical operating point.

Source code: <https://github.com/DavidLikesLearning/PythonPowerFlow/tree/DavidProject>